

The Voronoi diagram

Fortune's algorithm

| The problem

Geometric definition:

Given a set $P = \{p_1, \dots, p_n\}$ on n point sites in the plane, the goal is to partition the plane into n distinct cells such that:

- Every point x in cell $V(p_i)$ is geometrically closer to site p_i than to any other site p_j

Formal definition:

$$V(p_i) = \{x \in R^2 \mid \text{dist}(x, p_i) \leq \text{dist}(x, p_j) \quad \forall i \neq j\}$$

Goal:

Compute the the Voronoi diagram guaranteeing an optimal time complexity of $O(n \log n)$

Fortune's algorithm — pseudocode

Algorithm VORONOIDIAGRAM(P)

1. **Initialize** event queue Q with all site events, empty status structure T , and empty DCEL D .
2. **while** Q is not empty **do**
3. **Remove** event with largest y-coordinate from Q .
4. **if** the event is a site_event, at site p_i
5. then **HANDLESITEEVENT**(p_i)
6. else **HANDLECIRCLEEVENT**(γ)
7. Attach semi-infinite edges to a bounding box.
8. Traverse half-edges to build Face records.

Complexity: $O(n \log n)$

| Data structure — Point

The **Point** class represents a site or a vertex in the plane. It includes coordinate rounding for robust hashing and equality checks.

The `circumcenter` function uses Cramer's rule to find the center of a circle passing through three points, critical for circle events.

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __eq__(self, other):
        return abs(self.x - other.x) < 1e-9 \
            and abs(self.y - other.y) < 1e-9
    def __hash__(self):
        return hash((round(self.x, 9), round(self.y, 9)))

def circumcenter(a, b, c):
    A1 = 2 * (b.x - a.x); B1 = 2 * (b.y - a.y)
    C1 = b.x**2 + b.y**2 - a.x**2 - a.y**2
    A2 = 2 * (c.x - b.x); B2 = 2 * (c.y - b.y)
    C2 = c.x**2 + c.y**2 - b.x**2 - b.y**2
    denom = A1 * B2 - A2 * B1
    if abs(denom) < 1e-9: return None
    cx = (C1 * B2 - C2 * B1) / denom
    cy = (A1 * C2 - A2 * C1) / denom
    return Point(cx, cy)
```

| Data structure — Event Queue

Event	Trigger	Result
SiteEvent	Sweep line hits site	New beach line arc
CircleEvent	Three arcs converge	Voronoi vertex

```
class Event:
    SITE = 1; CIRCLE = 2
    def __init__(self, type_, point):
        self.type = type_
        self.point = point
        self.valid = True
    def __lt__(self, other):
        return (self.point.y > other.point.y) or (
            abs(self.point.y - other.point.y) < 1e-9
            and self.point.x < other.point.x)

class SiteEvent(Event):
    def __init__(self, point):
        super().__init__(Event.SITE, point)

class CircleEvent(Event):
    def __init__(self, point, arc, center, radius):
        super().__init__(Event.CIRCLE, point)
        self.arc = arc
        self.center = center
        self.radius = radius
```

The valid flag allows lazy deletion of false-alarm circle events. Tie-breaking prioritizes lower x values for equal y.

| Data structure — Beach line (AVL tree)

Stored as a balanced AVL tree where:

- **Leaves:** parabolic arcs
- **Internal Nodes:** breakpoints (intersection points)

```
class BeachNode:
    def __init__(self, site=None, is_leaf=False):
        self.is_leaf = is_leaf
        self.parent = self.left = self.right = None
        self.height = 1
        # leaf attributes (arcs)
        self.site = site
        self.event = None
        self.prev = self.next = None
        # internal attributes (breakpoints)
        self.left_site = None
        self.right_site = None
        self.edge = None
```

Operation	Role
find_arc_above	Binary search for specific arc
get_breakpoints	Intersection x of parabolas
rebalance	Maintain $O(\log n)$ tree depth

Data structure — DCEL

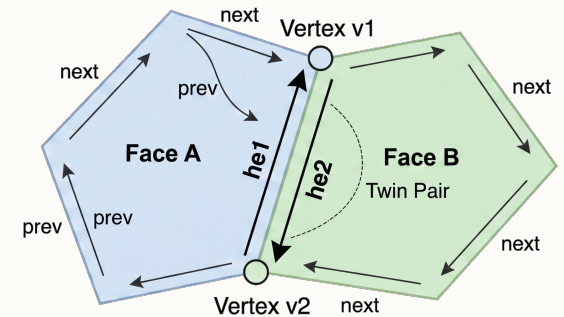
The **Doubly-Connected Edge List** is the standard output format. Every edge consists of a twin pair of half-edges.

- Vertex: stores geometric coordinates
- Face: corresponds to a site's Voronoi cell
- HalfEdge: traverses the cell boundary

```
class HalfEdge:
    def __init__(self, site=None):
        self.origin = None
        self.twin = None
        self.face = None
        self.next = None
        self.prev = None
        self.site = site
```

```
class Face:
    def __init__(self, site):
        self.site = site
        self.outer_component = None
```

```
class DCEL:
    def create_twin_pair(self, site_a, site_b):
        he_a = HalfEdge(site_a)
        he_b = HalfEdge(site_b)
        he_a.twin = he_b; he_b.twin = he_a
        return he_a, he_b
```



1 Edge Record = 2 Half-Edges

| Step 1 — Initialize Data Structures

Points are sorted by decreasing **y** coordinates (and increasing **x** as a tie-breaker).

The priority queue is pre-loaded with all site events before the sweep starts.

```
class FortuneVoronoi:
    def __init__(self, points):
        self.points = sorted(points,
                               key=lambda p: (-p.y, p.x))
        self.dcel = DCEL()
        self._beach = BeachLine()
        self._queue = EventQueue()
        for p in self.points:
            self._queue.push(SiteEvent(p))
```


| Steps 2-3 — Event loop

The main loop processes events one-by-one from the heap.

Circle events are often invalidated (marked as "false alarms") when the beach line structure changes before they are reached.

```
def compute(self):
    while not self._queue.is_empty():
        event = self._queue.pop()
        if not event.valid:
            continue
        if event.type == Event.SITE:
            self._handle_site(event)
        else:
            self._handle_circle(event)
    self._attach_to_bounding_box()
    self._build_faces()
    return self.dcel
```

Steps 4-5 — Site Event Handling

- 1 Find arc α above new site p_i
- 2 Split α into three arcs: α_{left} , α_{mid} , α_{right}
- 3 Create two new internal nodes (breakpoints)
- 4 Add half-edge pair to DCEL
- 5 Check for circle events between new neighbors

```
def _handle_site(self, event):
    p = event.point
    if self._beach.is_empty():
        self._beach.set_root(BeachNode(p, is_leaf=True))
        return
    arc = self._beach.find_arc_above(p, p.y)
    if arc.event is not None:
        arc.event.valid = False

    left_leaf = BeachNode(arc.site, is_leaf=True)
    mid_leaf = BeachNode(p, is_leaf=True)
    right_leaf = BeachNode(arc.site, is_leaf=True)

    # bp_left and bp_right
    bp_left = BeachNode(is_leaf=False)
    bp_left.left_site = arc.site; bp_left.right_site = p

    he_left, he_right = self.dcel.create_twin_pair(
        site_a=arc.site, site_b=p
    )
    bp_left.edge = he_left
```

Step 6 — Circle Event Handling

- 1 Identify triple of converging arcs
- 2 Create a Voronoi vertex at the circumcenter
- 3 Close three half-edges at the vertex
- 4 Remove the middle arc and rebalance the tree
- 5 Check for new circle events on left/right triples

```
def _handle_circle(self, event):
    arc = event.arc; left_arc = arc.prev; right_arc = arc.next
    vertex = self.dcel.create_vertex(event.center)

    # Close half-edges at the vertex
    he_left_face.twin.origin = vertex
    he_arc_face.twin.origin = vertex

    # Update beach line structure
    left_arc.next = right_arc; right_arc.prev = left_arc

    # Surviving breakpoint gets new half-edge pair
    he_nl, he_nr = self.dcel.create_twin_pair(
        site_a=left_arc.site, site_b=right_arc.site)
    he_nl.origin = vertex

    # Check new neighbor triples
    self._check_circle_event(left_arc.prev, left_arc, right_arc, ...)
```

| Step 7 — Bounding box clipping

Voronoi diagrams contain semi-infinite edges. To finalize the DCEL, we intersect these rays with a large bounding box.

- Define box limits based on site coordinates
- Direction is perpendicular to the site bisector
- Use ray-box intersection to create finite boundary vertices

```
def _attach_to_bounding_box(self):
    margin = 1000
    # xs, ys from sites and vertices
    min_x = min(xs) - margin; max_x = max(xs) + margin

    for he in self.dcel.half_edges:
        if he.origin is not None and he.twin.origin is not None:
            continue
        # Direction vector (fx, fy)
        fx = he.twin.site.y - he.site.y
        fy = he.site.x - he.twin.site.x
        isec = self._ray_box_intersection(known,
            Point(fx, fy), box_corners)
        if isec:
            he.origin = self.dcel.create_vertex(isec)
```

| Step 8 — Build face records

The final step populates the Face records in the DCEL by following the cycle of half-edges.

- Traverse each half-edge's next pointer chain
- Assign every half-edge in the cycle to the cell's Face
- Set the outer_component for the cell

```
def _build_faces(self):  
    visited = set()  
    for he in self.dcel.half_edges:  
        if he.face is not None or he in visited:  
            continue  
        face = self.dcel.create_face(he.site)  
        curr = he  
        while curr is not None and curr not in visited:  
            visited.add(curr)  
            curr.face = face  
            if face.outer_component is None:  
                face.outer_component = curr  
            curr = curr.next
```

Final output

